# 科协运营管理平台

设计报告

作品形式：Web 应用 + MCP 工具服务 + 微信小程序

技术栈：Node.js · Express · node:sqlite · React · Taro · Docker

文档版本：v1.0 | 日期：2026 年 11 月

# 1 项目概述

## 1.1 背景

学生科技协会的日常运营涉及设备与 3D 打印预约、耗材发放、仪器借还、实验室门禁、元器件库存、活动组织、值班排班、资料归档等十余类事务。实际工作中，这些环节通常分散在微信群、纸质登记表、Excel 表格和多个网盘之间，导致三类问题：

- **信息不透明**：设备是否空闲、耗材还剩多少、门禁谁批准，只能靠人逐个询问。
- **流程无留痕**：借用、审批、发放等关键动作没有统一记录，出现损坏或丢失难以追溯。
- **交接成本高**：负责人换届后，历史数据、权限边界、操作规范都难以完整移交。

本项目把上述业务收敛到一个统一平台，以「一套账号、一套权限、一套数据」覆盖科协运营全流程，并以多种形式对外提供能力：普通成员使用 Web 界面或微信小程序，管理者使用后台完成审批与统计，AI 助手可通过 MCP 工具服务查询运营数据。

## 1.2 目标

| 业务覆盖 | 角色模型 | 部署方式 | 对外能力 |
| --- | --- | --- | --- |
| **12 个业务模块** | **2 角色 + 8 权限** | **Docker 三容器** | **REST API / MCP** |

- 业务在线化：预约、借还、报名、值班等高频操作全部线上完成，状态实时可见。
- 权限可配置：管理员按成员职责勾选权限，敏感操作（记账、改设备、看统计）逐接口校验。
- 部署低成本：单机 Docker Compose 一键启动，无需额外数据库服务，适合社团自建服务器或 NAS。
- 数据可追溯：关键写操作写入审计表，备份可下载、可整目录迁移。

# 2 需求分析

## 2.1 用户与角色

| 角色 | 职责 | 典型操作 |
| --- | --- | --- |
| 管理员 | 平台维护、人员与权限管理、审批、数据导出 | 成员增删改、权限分配、预约审批、门禁审批、API 密钥签发、系统状态查看、数据备份 |
| 成员 | 日常使用，按被授予的权限扩展能力 | 设备预约、耗材领用、仪器借还、活动报名、查看值班、下载资料 |

成员在「成员」角色基础上可叠加 8 项可配置权限：`finance`（记账）、`equipment`（设备管理）、`inventory`（库存管理）、`access`（门禁审批）、`activity`（活动管理）、`duty`（值班管理）、`doc`（资料管理）、`api`（API 调用）。这一设计替代了早期「财务员 / 技术员」的硬编码角色，避免角色爆炸。

### 2.2 功能需求

| 模块 | 核心功能 | 关键约束 |
| --- | --- | --- |
| 设备与预约 | 设备台账、分类、状态（可用/维护/停用）、预约申请、审批、开始/完成/取消、使用记录 | 同一设备时间段不可重叠；需审批的设备先进入待审核 |
| 3D 打印 | 打印机作为设备类型管理，打印任务按预约流程处理 | 复用预约冲突检测 |
| 耗材 | 耗材台账、入库/出库、领用记录、低库存预警 | 库存不得为负；变动写流水 |
| 仪器借还 | 借出登记、归还确认、逾期标记 | 未归还不可重复借出 |
| 维护记录 | 设备保养/维修登记、费用与责任人 | 关联设备，写审计 |
| 门禁 | 门禁申请、审批、通行记录 | 审批人与申请人权限分离 |
| 元器件库存 | 物料台账、入库/领用、盘点、低库存预警、流水 | 数量与阈值校验 |
| 成员与权限 | 成员创建/编辑/停用/重置密码、权限勾选、批量导入 | 仅管理员；不可删除自身 |
| 活动 | 活动发布、报名/取消、人数上限、报名截止、签到、报名名单导出 | 满员或截止后拒绝报名 |
| 值班 | 班次创建、人员分配、签到、在岗状态实时更新、今日值班展示 | 同班次人员去重 |
| 资料归档 | 文档上传、分类、检索、下载 | 类型与大小白名单 |
| 统计与导出 | 运营总览、设备台账导出、账单导出、审计日志查询 | 按权限过滤数据范围 |

### 2.3 非功能需求

- **安全**：口令不可逆存储、登录失败限流、接口级鉴权、全面审计、HTTPS 部署。
- **可部署性**：一条命令启动；数据目录可整体迁移；支持 x86 与 ARM（NAS）。
- **可维护性**：前后端分离、路由按业务拆分、统一错误处理与参数校验。
- **可扩展性**：对外提供 REST API 与 MCP 工具，便于接入 AI 助手或其他系统。

## 3 总体设计

### 3.1 系统架构






三个进程通过同一数据目录协作：Web 与 API 由应用进程处理，备份与定时任务交给独立的 worker 进程，MCP 服务以只读方式调用 REST API。容器化部署时三者由 Docker Compose 编排，Nginx 容器仅对宿主机暴露一个端口。

### 3.2 技术选型

| 层次 | 选型 | 理由 |
| --- | --- | --- |
| 运行时 | Node.js 22.5+ | 内置 `node:sqlite`，无需编译原生模块，镜像构建稳定 |
| Web 框架 | Express 4 | 生态成熟、中间件模型简单，适合中小型业务系统 |
| 数据库 | SQLite（WAL 模式） | 单机零运维、单文件易备份；社团量级完全够用 |
| 前端 | React 18 + Vite 6 | 组件化开发、构建快、产物可由 Nginx 直接托管 |
| 小程序 | Taro 4 + TypeScript | 一套代码可编译到微信小程序与 H5，便于预览验证 |
| 部署 | Docker Compose | 环境一致、一键启停、健康检查与自动重启 |
| 测试 | Vitest + Supertest | 直接对 HTTP 接口做端到端断言，覆盖鉴权与业务规则 |

### 3.3 总体方案

系统采用「一个后端、多端接入、按业务垂直拆分」的总体方案：后端以 Express 提供统一的 REST API，鉴权、参数校验、错误处理、审计下沉为公共中间件，业务按设备、库存、门禁、活动、值班、资料、经费、系统八条路由线拆分；Web 前端与微信小程序复用同一套接口，MCP 服务在接口之上做只读封装。由此保证三端看到的权限边界、数据口径与审计记录完全一致。

实现路径按依赖关系推进：先完成账号、权限与审计骨架，再实现设备预约（含冲突检测）、耗材与元器件库存、仪器借还，随后补齐门禁、活动、值班、资料与经费台账，最后完成统计导出、API Key、MCP 工具与容器化部署。每一步都以可运行的接口和测试用例作为完成标准，避免「先堆界面后补逻辑」。

| 交付形态 | 面向对象 | 能力边界 |
| --- | --- | --- |
| Web 应用 | 社团成员、管理员 | 全部业务操作与后台管理 |
| 微信小程序 | 移动端成员 | 登录、预约、库存查询、活动报名、资料下载 |
| MCP 工具服务 | AI 助手 / Agent | 七类只读查询，写操作仍回到 Web 界面 |
| REST API + API Key | 第三方系统 | 按持有人权限开放全部业务接口 |

# 4 数据设计

## 4.1 核心数据表

| 表 | 主要内容 | 说明 |
| --- | --- | --- |
| `users` | 账号、口令哈希、角色、权限数组、状态、token_version | 权限以 JSON 数组存储，便于扩展 |
| `transactions` / `categories` | 收支流水、分类、附件 | 保留班费记账能力 |
| `equipment` / `equipment_bookings` | 设备台账、预约时间段、审批状态、用途 | 冲突检测依赖时间段区间比较 |
| `consumables` / `consumable_logs` | 耗材库存与出入库流水 | 库存变动成对写入 |
| `inventory_items` / `inventory_logs` | 元器件库存、预警阈值、流水 | 台账与流水分离 |
| `borrows` | 借出人、设备/仪器、借出与归还时间 | 未归还记录用于逾期提醒 |
| `access_requests` | 门禁申请、审批人、结果 | 审批留痕 |
| `activities` / `activity_signups` | 活动信息、报名状态、签到时间 | 人数上限在事务内校验 |
| `duty_shifts` / `duty_assignments` | 班次时间地点、值班人员 | 支持多人同班 |
| `documents` | 文件名、分类、大小、上传人 | 文件落盘，元数据入库 |
| `audit_logs` | 操作人、动作、对象、IP、UA、时间 | 事后追溯依据 |
| `api_keys` | 密钥哈希、前缀、有效期、撤销状态 | 明文不落库 |
| `login_failures` | 来源+账号、失败次数、时间窗 | 限流状态持久化 |

设计原则：台账表只存「当前值」，所有数量变化另写流水表，既保证查询快，又能还原历史。删除类操作以状态字段代替物理删除（设备停用、成员禁用、物料归档），避免破坏历史关联。

### 4.2 数据存储

数据库文件位于数据目录下的 `classfund.db`，开启 WAL 模式以支持读写并发；上传文件存 `uploads/`，每日备份存 `backups/`。生产部署将该目录挂载进容器，升级镜像不影响数据。

## 5 功能设计

### 5.1 设备预约（含 3D 打印）

成员选择设备、填写时间段与用途提交预约。系统先校验设备状态与时间合法性，再检查该设备是否存在时间重叠的有效预约；重叠则返回 409 并提示冲突时段。需审批设备进入「待审核」，管理员审批通过后方可使用。使用结束由设备管理员标记完成，或由系统按结束时间自动关闭。

> **冲突判定**：两条预约重叠的充要条件是 `已有开始 < 新结束 且 已有结束 > 新开始`。该判断由数据库执行，避免把全量数据拉到应用层过滤。

### 5.2 库存与借还

耗材与元器件均采用「台账 + 流水」模型。入库、领用、盘点都写一条流水，台账数量在事务内更新，任何一步失败整体回滚。数量低于预警阈值时在首页与库存页高亮，并在运营总览中计数。

### 5.3 活动与值班

活动支持草稿、开放、关闭、结束、取消五种状态，报名受截止时间与人数上限双重约束，满员返回明确提示；报名与取消均写审计。值班按班次组织，可指定多人、填写地点与容量，首页展示「今日值班」及「我是否在班」。在岗状态支持在岗、暂离、离岗三种取值：本人可在值班列表中实时切换，切换结果写入 `duty_assignments.presence` 与 `presence_at` 并记入审计；值班页每 10 秒静默轮询一次，同一页面上所有人在岗情况无需手动刷新即可保持同步，首页「今日值班」同时展示各人的在岗标记。

### 5.4 成员与权限

管理员创建成员时指定角色与权限；编辑时按需勾选八项能力。权限判断集中在服务端中间件：请求进入业务逻辑前先解析身份，再比对所需权限键。前端只负责隐藏入口，不作为安全边界。成员被停用或改密时，其已签发令牌通过版本号机制立即失效。

建号提供三种方式：单个新建、按起始账号批量生成、**Excel 名单导入**。名单导入时系统按「登录账号 → 学号 → 自动分配 kx0001 起账号」的顺序确定用户名，为每人生成符合复杂度要求的随机初始密码并强制首登改密；权限列支持中文写法（如「设备、库存、值班」），服务端解析为权限键。重复账号、缺姓名、权限无法识别等异常逐行跳过并返回行号与原因，导入结果可下载账号表分发，审计日志仅记录账号不记录密码。

### 5.5 资料与导出

资料按分类归档，上传时校验扩展名与大小上限，下载走鉴权接口而非公开静态目录。导出支持设备台账（全部或按类型）、账单与审计日志，输出 CSV/Excel，中文编码处理避免乱码。

## 6 关键技术实现

### 6.1 身份认证与令牌吊销

登录成功后签发 HS256 JWT，有效期 7 天。令牌可经三种方式携带：httpOnly Cookie（网页）、`Authorization: Bearer`（小程序与 API 调用）、查询参数（文件下载场景）。服务端不维护会话表，但每次鉴权都会读取用户当前状态与 `token_version`，与令牌载荷比对；改密、重置、停用都会使版本号加一，从而让旧令牌全部失效。首次登录强制修改初始密码。

### 6.2 登录限流

以「来源 IP + 账号」为键统计失败次数，15 分钟内达到 5 次即拒绝登录并提示稍后再试。计数写入数据库而非内存，进程重启后依然生效，避免攻击者通过重启绕过。成功登录后清除计数。

### 6.3 API 密钥

管理员或具备 `api` 权限的成员可签发密钥，明文仅在创建时展示一次，数据库只存 SHA-256 摘要。调用时通过 `X-API-Key` 或 `Authorization: ApiKey` 携带；密钥继承持有人权限，且必须满足 `api` 权限；单密钥按分钟限流，撤销或过期立即失效。

### 6.4 MCP 工具服务

为满足「以 MCP 工具服务形式交付」的要求，项目在不引入额外依赖的前提下实现了一个 MCP stdio 服务，通过 JSON-RPC 处理 `initialize`、`tools/list`、`tools/call`，将设备、库存、耗材、活动、预约、运营总览、资料检索七类查询封装为工具。工具内部复用 REST API 与 API Key 鉴权，因此无需在 MCP 层重复实现权限体系；所有工具均为只读，写操作仍回到 Web 界面，降低 Agent 误操作风险。

### 6.5 容器化与进程协作

镜像采用多阶段构建：先构建前端产物，再安装服务端生产依赖，最终镜像同时包含静态资源与 API 服务。Compose 定义应用、worker、Nginx 三个服务，应用与 worker 共享数据卷，Nginx 依赖应用健康检查通过后启动，从而避免「容器起来了但接口未就绪」。

# 7 安全设计

| 威胁 | 对策 | 实现位置 |
| --- | --- | --- |
| 口令泄露 | BCrypt（cost 10）单向哈希；初始密码强制修改 | 账号模块 |
| 暴力破解 | IP+账号 15 分钟 5 次锁定，计数持久化 | 登录接口 |
| 令牌被盗 | HTTPS；httpOnly + SameSite；token_version 即时吊销 | 鉴权中间件 |
| 越权访问 | 逐接口权限中间件；管理员专属接口二次校验 | 全部写接口 |
| SQL 注入 | 全部使用参数化查询，禁止拼接用户输入 | 数据访问层 |
| 恶意上传 | 扩展名白名单 + 单文件大小上限 + 鉴权下载 | 资料/附件模块 |
| 密钥外泄 | API Key 只存摘要、可撤销、按分钟限流 | API 管理 |
| 跨站请求 | CORS 来源白名单；安全响应头（nosniff、SAMEORIGIN、Referrer-Policy） | 应用入口 |
| 事后追责 | 写操作记录操作人、动作、对象、IP、UA | 审计表 |

已知边界：未接入 WAF 与双因素认证；单机部署不具备异地容灾；DDoS 依赖云厂商防护；密钥文件需管理员妥善保管。

# 8 部署与运维

## 8.1 部署结构

- 宿主机安装 Docker 与 Compose，项目目录包含 `docker-compose.yml` 与数据目录。
- 应用容器监听内部端口，仅由 Nginx 容器访问；Nginx 对外暴露单一端口。
- 如需 HTTPS，由宿主机 Nginx 或云负载均衡终止 TLS 后转发至容器端口。
- 支持 NAS 部署：本地磁盘目录挂载数据卷，避免使用网络文件系统（SQLite 依赖文件锁）。

## 8.2 升级与备份

- 升级：拉取代码后执行 `docker compose up -d --build`，数据卷不受影响。
- 备份：worker 每日生成备份包并保留最近 14 天，管理员可一键立即备份并下载。
- 恢复：停止服务，替换数据目录，重启即可；数据库与上传文件同目录，保证一致性。
- 可观测：管理员「运行状态」展示版本、运行时长、内存、数据库体积、worker 在线状态与最近备份时间。

## 8.3 交付物与复现清单

为便于评测人员直接部署与验证，仓库根目录提供以下交付物，均已在公开仓库中可获取：

| 交付物 | 位置 | 说明 |
| --- | --- | --- |
| 作品源代码 | `server/`、`web/`、`miniapp/`、`mcp/` | 后端、前端、小程序、MCP 服务全部源码 |
| Dockerfile | `Dockerfile` | 多阶段构建，产出含静态资源与 API 的运行镜像 |
| Docker Compose 配置 | `docker-compose.yml` | app / worker / nginx 三服务编排，含健康检查 |
| 部署说明 | `README.md`「快速开始」「配置项」 | 环境变量、启动步骤、端口与数据目录 |
| 使用说明 | `README.md`、`docs/项目手册.md` | 功能入口、权限模型、外部 API 与 MCP 用法 |
| 示例数据 | `npm run seed:demo` | 六个演示账号与设备、耗材、预约、活动、账单等样例数据 |
| 测试方法与接口 | `tests/api.test.js`、README「评测验证指引」 | 自动化用例、健康检查、运行状态、审计日志查询入口 |
| 许可证 | `LICENSE` | Apache License 2.0 |

### 8.4 评测环境可用性说明（HTTPS 与微信小程序）

**HTTPS：**域名 `astmgr.icu` 的 Let's Encrypt 证书已签发（有效期至 2027-02），宿主机 Nginx 443 已配置 TLS 与 HTTP 跳转。因 **ICP 备案在提交截止前来不及完成**，域名链路在访问时被拦截，评测环境临时改用 `http://1.14.125.216/` 直连。此模式下口令与令牌在链路上未加密，仅用于评测；备案完成后改回域名并将 `COOKIE_SECURE` 置为 1 即可恢复全加密，代码无需改动。
**微信小程序：**因同样的备案原因无法配置 request 合法域名，且小程序未提交发布，**本次评测暂不可用。**小程序源码（Taro + React + TS）与 H5 预览完整保留在 `miniapp/`，复用同一套 REST API 与 Bearer 鉴权；备案完成后修改 `miniapp/src/config.ts` 中的 `API_BASE_URL` 为 HTTPS 域名即可启用，服务端无需改动。

## 9 测试与验证

### 9.1 测试方案与测试环境

测试分三层：**接口自动化测试**覆盖鉴权、权限与业务规则；**场景化验证**针对并发、越权、限流等难以用单条断言描述的路径，用脚本与多账号实测；**部署验证**在真实服务器上用容器化方式启动并逐项检查。

| 项目 | 内容 |
| --- | --- |
| 测试框架 | Vitest 3 + Supertest 7，直接对 Express 实例发起 HTTP 请求 |
| 运行方式 | `npm test`；`tests/setup.js` 在加载应用前把数据目录指向系统临时目录，不影响 `server/data` |
| 用例规模 | 42 个自动化用例 + 6 个场景化验证项 + 部署验证清单 |
| 验证环境 | Linux（Ubuntu）+ Docker 24 / Compose v2，Node.js 24 容器内运行，SQLite WAL |
| 判定标准 | 用例全部通过；异常路径返回预期状态码（401/403/409/429）；审计表有对应记录 |

### 9.2 自动化测试

测试基于 Vitest + Supertest，直接对 HTTP 接口断言，共 41 个用例，覆盖：登录成功/失败与限流、未登录访问拒绝、越权访问返回 403、成员权限配置生效、**成员名单导入（账号自动分配、初始密码规则、首登改密、重复账号跳过、未登录拒绝）**、设备预约与冲突拒绝、库存出入库与预警、活动报名上限、资料上传类型校验、API Key 签发与鉴权、CORS 预检、系统状态仅管理员可见等。全部用例通过（41/41）。

### 9.3 关键场景验证

| 场景 | 方法 | 结果 |
| --- | --- | --- |
| 并发预约同一设备同一时段 | 两个账号同时提交重叠预约 | 一个成功、一个 409 |
| 普通成员直接调用管理接口 | 使用成员令牌请求成员删除接口 | 403，且写入审计 |
| 登录暴力破解 | 连续错误 5 次后重试 | 锁定并提示，重启后仍生效 |
| 改密后旧令牌 | 改密前保存令牌，改密后继续使用 | 401，需重新登录 |
| API Key 撤销 | 撤销后再次调用 | 401 |
| 容器化部署 | 服务器上构建启动并访问健康检查 | 健康检查 200，页面与接口正常 |

### 9.4 部署验证结果

- `docker compose ps`：app、worker、nginx 三容器均为 healthy，`restart: unless-stopped` 生效。
- `GET /api/v1/health`：返回 `{"ok":true,"version":"1.0.0"}`。
- 演示账号登录：admin、finance01、equip01、tech01、access01、media01 六个账号全部返回 200，错误口令返回 401。
- `GET /api/v1/system/status`：管理员可获取版本、运行时长、内存、数据库体积、worker 在线状态与最近备份。
- `GET /api/v1/logs`：可检索到登录、审批、导出等审计记录；普通成员访问返回 403。

# 10 创新点与展望

## 10.1 创新点

- **一平台多形态**：同一套后端同时服务 Web、小程序与 MCP 工具，权限与审计口径统一。
- **权限可配置而非角色堆叠**：两角色 + 八权限的组合，比传统「财务员/技术员/管理员」更贴合社团人员流动。
- **零外部依赖的持久化方案**：内置 SQLite + 单文件备份，让社团无需数据库运维能力也能长期运行。
- **可解释的安全设计**：令牌吊销、限流、密钥摘要等机制均有明确的威胁对应关系，便于评审与交接。

## 10.2 不足与改进方向

- 目前为单机单实例，无高可用；后续可引入对象存储做异地备份，并增加监控告警。
- 未接入统一身份认证（学校 SSO）与双因素认证，可作为下一阶段安全增强。
- MCP 工具当前只读，后续可在审批场景引入「人工确认后执行」的写操作工具。
- 小程序端可继续补充扫码借还、消息订阅提醒等移动端特性。

# 11 代码结构与自主完成说明

## 11.1 代码组织与自研范围

| 目录 | 内容 | 自主完成 |
| --- | --- | --- |
| `server/src/app.js`、`db.js`、`auth.js` | 应用装配、建表与迁移、事务封装、鉴权与权限中间件 | 全部自研 |
| `server/src/routes/` | 设备、预约、耗材、元器件、借还、门禁、活动、值班、资料、经费、成员、API Key、系统、日志等路由 | 全部自研 |
| `server/src/worker.js`、`backup.js` | 定时维护任务与备份生成 | 全部自研 |
| `server/src/seed.js` | 演示账号与示例数据生成 | 全部自研 |
| `web/src/` | React 页面、组件、接口封装与路由 | 全部自研 |
| `miniapp/src/` | Taro 小程序页面、请求封装与登录态管理 | 全部自研 |
| `mcp/server.mjs` | MCP stdio 服务与七个只读工具 | 全部自研，未依赖 MCP 官方 SDK |
| `tests/api.test.js` | 接口自动化测试 | 全部自研 |
| `Dockerfile`、`docker-compose.yml`、`deploy/` | 镜像构建、服务编排、Nginx 配置 | 全部自研 |

以上自研代码随报告一并提交，完整可运行，不依赖任何未公开的私有模块。

### 11.2 第三方组件、开源项目与外部服务

| 名称 | 许可证 | 用途与使用范围 | 来源 |
| --- | --- | --- | --- |
| Express 4 | MIT | HTTP 框架，承载全部 REST 接口；未修改源码 | `expressjs.com` |
| bcryptjs 3 | BSD-3-Clause | 口令单向哈希（cost 10） | `github.com/dcodeIO/bcrypt.js` |
| jsonwebtoken 9 | MIT | HS256 令牌签发与校验 | `github.com/auth0/node-jsonwebtoken` |
| cookie-parser 1 | MIT | Cookie 解析，用于 Web 端令牌读取 | `github.com/expressjs/cookie-parser` |
| multer 2 | MIT | 资料与附件上传 | `github.com/expressjs/multer` |
| exceljs 4 | MIT | 台账、账单、审计日志的 Excel 导入导出 | `github.com/exceljs/exceljs` |
| archiver 7 | MIT | 多文件 ZIP 打包导出 | `github.com/archiverjs/node-archiver` |
| React 18 / React Router 6 | MIT | 前端组件与路由 | `react.dev`、`reactrouter.com` |
| Vite 6 | MIT | 前端开发与构建 | `vite.dev` |
| Vitest 3 / Supertest 7 | MIT | 接口测试与断言 | `vitest.dev`、`github.com/ladjs/supertest` |
| ESLint 10 / Prettier 3 | MIT | 代码检查与格式化，仅开发期使用 | `eslint.org`、`prettier.io` |
| Node.js 24 | MIT | 运行时；内置 `node:sqlite` 提供数据库能力，未引入原生扩展 | `nodejs.org` |
| Nginx 1.27（Alpine 镜像） | BSD-2-Clause | 反向代理与静态资源托管 | `nginx.org` |
| Alpine Linux | MIT 等 | 容器基础镜像 | `alpinelinux.org` |
| 微信小程序登录接口 | 腾讯服务条款 | **可选**外部服务：仅在配置 `WX_APPID` / `WX_SECRET` 时用于 `code2session` 免密登录；未配置则完全不走该链路 | `developers.weixin.qq.com` |

上述组件均按其原始许可证使用，未修改其源代码，完整许可证文本见各组件的发行包。项目自身以 Apache License 2.0 发布。

### 11.3 自动生成代码与 AI 辅助说明

- 项目未使用代码生成器批量产出业务代码；`package-lock.json` 由 npm 自动生成，属于依赖锁定文件，不属于作品逻辑。
- 开发过程中使用 AI 编程助手进行**辅助编写、代码审查与文档润色**，所有生成内容均经团队成员逐行阅读、修改、运行验证后纳入代码库，团队对全部代码的行为与质量负责。
- 未引入任何预训练模型权重、第三方数据集或商业 AI 服务参与运行时逻辑；系统中的全部演示数据由 `seed.js` 脚本生成，不含真实个人信息。
- 作品运行时不调用除微信登录（可选）以外的任何外部服务，数据完全保存在本地 SQLite 与文件目录中。

## 附录 A 演示账号与验证入口

执行 `docker compose exec app npm run seed:demo` 后，以下账号可直接登录（浏览器访问 `http://1.14.125.216/` 或本机 `http://localhost:8080`），统一密码 `Kx@2026demo`，且不强制首次改密：

| 账号 | 角色 / 权限 | 验证内容 |
| --- | --- | --- |
| `admin` | 管理员 | 成员与权限配置、各类审批、运行状态、审计日志、API 密钥、数据备份 |
| `finance01` | 成员 + 经费（收入/支出） | 收支记账、账单导出、凭证审核 |
| `equip01` | 成员 + 设备、库存 | 设备台账、预约审批、耗材与元器件出入库 |
| `tech01` | 成员 + 设备、库存、值班、API | 设备维护记录、值班排班、API Key 签发与外部调用 |
| `access01` | 成员 + 门禁、值班 | 门禁申请与审批、通行记录 |
| `media01` | 成员 + 活动、资料 | 活动发布与报名、资料上传与下载 |

复查入口：`GET /api/v1/health`（免登录）、`GET /api/v1/system/status`（管理员）、`GET /api/v1/logs`（管理员）。若需恢复初始状态，删除数据目录后重新启动并再次执行 seed 脚本。

## 附录 B 主要接口

| 接口 | 说明 |
| --- | --- |
| `POST /api/v1/auth/login` | 账号密码登录，返回令牌与用户信息 |
| `GET /api/v1/summary/operations` | 运营总览：待审核、门禁、活动、低库存、今日值班 |
| `GET/POST /api/v1/equipment`、`/equipment/bookings` | 设备台账与预约（含审批、取消、完成） |
| `GET/POST /api/v1/inventory`、`/inventory/:id/stock` | 元器件库存与出入库 |
| `GET/POST /api/v1/activities`、`/activities/:id/signup` | 活动与报名 |
| `GET/POST /api/v1/documents` | 资料归档与下载 |
| `GET /api/v1/system/status` | 运行状态（仅管理员） |
| `POST /api/v1/api-keys` | 签发 API 密钥 |